LAPORAN PRAKTIKUM PEKAN 3

STRUKTUR DATA

Disusun Oleh:

Syasya Halwa Gazwani

(2511531018)

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid Alvero

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Assalamu'alaikum warahmatullahi wabarakatuh,

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Struktur Data, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW. yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini. Penulis mengucapkan terima kasih kepada bapak Dr. Wahyudi, S.T, M.T. selaku dosen mata kuliah Struktur Data yang telah memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan sesuai dengan mata kuliah yang penulis tekuni. Penulis juga mengucapkan terima kasih kepada semua pihak yang telah membagi sebagian pengetahuannya sehingga penulis dapat menyelesaikan Laporan Praktikum ini.

Padang, April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR i

DAFTAR ISI ii

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan 1

1.3 Manfaat Praktikum 1

BAB II PEMBAHASAN 3

2.1 QueueArray_2511531018 3

2.1.1 Kode Program 3

2.1.2 Langkah Kerja 4

2.2 QueueArrayDriver_2511531018 6

2.2.1. Kode Program 6

2.2.2. Langkah Kerja 7

2.2.3 Output 8

2.3 QueueLinkedList_2511531018 8

2.3.1 Kode Program 8

2.3.2 Langkah Kerja 8

2.4.3 Output 9

2.4 ReverseData_2511531018 10

2.4.1 Kode Program 10

2.4.2 Langkah Kerja 10

2.4.3 Output 11

2.5 IterasiQueue_2511531018 11

2.5.1 Kode Program 11

2.5.2 Langkah Kerja 11

2.5.5 Output 12

2.6 Tugas..... 12

2.6.1 Kode Program AntrianLoket_2511531018 12

2.6.2 Output 15

BAB III PENUTUP..... 17

DAFTAR PUSTAKA..... 18

BAB I PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, struktur data memiliki peran penting dalam mengelola dan menyusun data secara efisien. Salah satu struktur data yang sering digunakan adalah Queue (antrian), yang bekerja dengan prinsip FIFO (First In First Out), di mana data yang pertama masuk akan menjadi data pertama yang keluar. Konsep ini banyak diterapkan dalam kehidupan sehari-hari, seperti antrian di loket pelayanan, kasir, maupun sistem antrean digital. Oleh karena itu, diperlukan pemahaman mengenai cara kerja dan implementasi Queue dalam bahasa pemrograman, khususnya menggunakan array maupun struktur bawaan seperti LinkedList, agar dapat digunakan dalam menyelesaikan permasalahan secara sistematis.

1.2 Tujuan

1.2.1 Memahami konsep dasar struktur data Queue (antrian).

1.2.2 Mengimplementasikan Queue menggunakan array dalam bahasa Java.

1.2.3 Mengetahui operasi dasar Queue seperti enqueue, dequeue, display, isEmpty, dan isFull.

1.2.4 Memahami penggunaan Queue menggunakan LinkedList.

1.2.5 Mengimplementasikan fungsi tambahan seperti reverse dan iterasi pada Queue.

1.2.6 Membuat program antrian loket yang interaktif menggunakan menu.

1.3 Manfaat Praktikum

1.3.1 Meningkatkan pemahaman tentang struktur data Queue dan penerapannya.

1.3.2 Melatih kemampuan dalam mengimplementasikan struktur data menggunakan bahasa Java.

1.3.3 Membantu memahami pengelolaan data secara terstruktur dan sistematis.

1.3.4 Memberikan gambaran penerapan Queue dalam kehidupan nyata, seperti sistem antrian pelayanan.

1.3.5 Menjadi dasar untuk mempelajari struktur data yang lebih kompleks di masa mendatang.

BAB II PEMBAHASAN

2.1 QueueArray_2511531018

2.1.1 Kode Program

```
package pekan4_2511531018;

public class QueueArray_2511531018 {
   int front_1018, rear_1018, size_1018;
   int capacity_1018;
   int array_1018[];

   public QueueArray_2511531018(int capacity) {
         this.capacity_1018 = capacity;
         front_1018 = this.size_1018 = 0;
         rear_1018 = capacity - 1;
         array_1018 = new int[this.capacity_1018];

   }
    boolean isFull_1018(QueueArray_2511531018 queue_1018) {
          return (queue_1018.size_1018 ==
queue_1018.capacity_1018);
    }
    boolean isEmpty_1018(QueueArray_2511531018 queue_1018) {
          return (queue_1018.size_1018 == 0);
    }
    void enqueue_1018(int item) {
          if (isFull_1018(this))
                return;
          this.rear_1018 = (this.rear_1018 + 1) %
this.capacity_1018;
          this.array_1018[this.rear_1018] = item;
          this.size_1018 = this.size_1018 + 1;
          System.out.println(item + " enqueued to queue");
    }
    int dequeue_1018() {
          if (isEmpty_1018(this))
                return Integer.MIN_VALUE;
          int item = this.array_1018[this.front_1018];
          this.front_1018 = (this.front_1018 + 1) %
this.capacity_1018;
          this.size_1018 = this.size_1018 - 1;
          return item;
    }
    int front_1018() {
          if (isEmpty_1018(this))
                return Integer.MIN_VALUE;

          return this.array_1018[this.front_1018];
    }
    int rear_1018() {
          if (isEmpty_1018(this))
                return Integer.MIN_VALUE;
          return this.array_1018[this.rear_1018];
    }
```

```
        //mencetak elemen antrian
    void display_1018() {
            int i_1018;
            if (front_1018 == rear_1018) {
                    System.out.printf("\nAntrian Kosong\n");
                    return;
            }
                    //kunjungi dari belakang dan cetak
                    for (i_1018 = front_1018; i_1018 < rear_1018;
i_1018++) {
                            System.out.printf(" %d <-- ",
array_1018[i_1018]);
                    }
                    return;
    }
}
```

Gambar 2.1.1 Kode program QueueArray_2511531018.

2.1.2 Langkah Kerja

2.1.2.1 Front_1018 => petunjuk elemen depan antrian, rear_1018 => petunjuk elemen belakang antrian, size_1018 => jumlah elemen dalam queue, capacity_1018 => kapasitas maksimal queue, dan array_1018[] => tempat menyimpan data.

```
public class QueueArray_2511531018 {
  int front_1018, rear_1018, size_1018;
  int capacity_1018;
  int array_1018[];
```

Gambar 2.1.2.1 Deklarasi *class* dan variabel.

2.1.2.2 Capacity diisi dari parameter, front=0 => antrian mulai dari indeks 0, size=0 => awalnya kosong, rear=capacity-1 => supaya saat enqueue pertama jadi 0, dan array dibuat sesuai kapasitas.

```
public QueueArray_2511531018(int capacity) {
        this.capacity_1018 = capacity;
        front_1018 = this.size_1018 = 0;
        rear_1018 = capacity - 1;
        array_1018 = new int[this.capacity_1018];
```

Gambar 2.1.2.2 Constructor (inisialisasi).

2.1.2.3 Jika jumlah elemen = kapasitas => penuh.

```
boolean isFull_1018(QueueArray_2511531018 queue_1018)
{
```

```
        return (queue_1018.size_1018 ==
queue_1018.capacity_1018);
```

Gambar 2.1.2.3 Cek queue penuh.

2.1.2.4 Jika size = 0 => kosong.

```
boolean isEmpty_1018(QueueArray_2511531018 queue_1018)
{
        return (queue_1018.size_1018 == 0);
```

Gambar 2.1.2.4 Cek queue kosong.

2.1.2.5 Mengecek apakah penuh atau tidak, rear = (rear+1) % capacity => circular, menyimpan item diposisi rear, dan menambah size.

```
  void enqueue_1018(int item) {
        if (isFull_1018(this))
              return;
        this.rear_1018 = (this.rear_1018 + 1) %
this.capacity_1018;
        this.array_1018[this.rear_1018] = item;
        this.size_1018 = this.size_1018 + 1;
        System.out.println(item + " enqueued to
queue");
  }
```

Gambar 2.1.2.5 Enqueue (menambah data).

2.1.2.6 Mengecek apakah kosong atau tidak, ambil data dari depan, menggeser front ke depan, dan mengurangi size.

```
int dequeue_1018() {
        if (isEmpty_1018(this))
              return Integer.MIN_VALUE;
        int item = this.array_1018[this.front_1018];
        this.front_1018 = (this.front_1018 + 1) %
this.capacity_1018;
        this.size_1018 = this.size_1018 - 1;
        return item;
  }
```

Gambar 2.1.2.6 Dequeue (menghapus data).

2.1.2.7 Mengambil nilai depan tanpa menghapus.

```
int front_1018() {
        if (isEmpty_1018(this))
              return Integer.MIN_VALUE;

        return this.array_1018[this.front_1018];
  }
```

Gambar 2.1.2.7 Mengambil elemen depan.

2.1.2.8 Mengambil nilai paling belakang.

```
int rear_1018() {
        if (isEmpty_1018(this))
                return Integer.MIN_VALUE;
        return this.array_1018[this.rear_1018];
    }
```

Gambar 2.1.2.8 Mengambil elemen belakang.

2.1.2.9 Jika front == rear dianggap kosong, loop dari depan ke belakang, dan menampilkan isi queue.

```
void display_1018() {
        int i_1018;
        if (front_1018 == rear_1018) {
                System.out.printf("\nAntrian
Kosong\n");
                return;
        }
            //kunjungi dari belakang dan cetak
            for (i_1018 = front_1018; i_1018 <
rear_1018; i_1018++) {
                System.out.printf(" %d <-- ",
array_1018[i_1018]);
            }
            return;
```

Gambar 2.1.2.9 Menampilkan isi queue.

2.2 QueueArrayDriver_2511531018

2.2.1. Kode Program

```
package pekan4_2511531018;

public class QueueArrayDriver_2511531018 {
   public static void main(String[] args) {
        QueueArray_2511531018 queue_1018 = new
QueueArray_2511531018(1000);
        queue_1018.enqueue_1018(10);
        queue_1018.enqueue_1018(20);
        queue_1018.enqueue_1018(30);
        queue_1018.enqueue_1018(40);
        System.out.println("Item di depan "+
queue_1018.front_1018());
        System.out.println("Item paling belakang "+
queue_1018.rear_1018());
        System.out.println("tampilan queue");
        queue_1018.display_1018();
        System.out.println();
        System.out.println(queue_1018.dequeue_1018() + "
dihapus dari queue");
        System.out.println("item di depan:" +
queue_1018.front_1018());
        System.out.println("item dibelakang: " +
queue_1018.rear_1018());
```

```
        System.out.println("tampilan queue setelah satu data
dihapus");
        queue_1018.display_1018();
    }
}
```

Gambar 2.2.1 Kode program QueueArrayDriver_2511531018.

2.2.2. Langkah Kerja

2.2.2.1 Membuat objek queue_1018 dengan kapasitas 1000 elemen.

```
QueueArray_2511531018 queue_1018 = new
QueueArray_2511531018(1000);
```

Gambar 2.2.2.1 Membuat objek Queue.

2.2.2.2 Memasuki data ke queue.

```
queue_1018.enqueue_1018(10);
queue_1018.enqueue_1018(20);
queue_1018.enqueue_1018(30);
queue_1018.enqueue_1018(40);
```

Gambar 2.2.2.2 Menambahkan data (Enqueue).

2.2.2.3 10 masuk lebih dulu dan 40 terakhir masuk. Menampilkan isi Queue.

```
System.out.println("Item di depan "+
queue_1018.front_1018());
        System.out.println("Item paling belakang "+
queue_1018.rear_1018());
System.out.println("tampilan queue");
queue_1018.display_1018();
```

Gambar 2.2.2.3 Menampilkan elemen depan dan belakang, lalu menampilkan isi Queue.

2.2.2.4 Depan => 20 dan belakang => 40.

```
System.out.println(queue_1018.dequeue_1018() + "
dihapus dari queue");
        System.out.println("item di depan:" +
queue_1018.front_1018());
        System.out.println("item dibelakang: " +
queue_1018.rear_1018())
```

Gambar 2.2.2.4 Menghapus data (dequeue) dan mengecek depan dan belakang.

2.2.2.5 Menampilkan queue setelah dequeue.

```
System.out.println("tampilan queue setelah satu data
dihapus");
        queue_1018.display_1018();
```

Gambar 2.2.2.5 Menampilkan queue.

2.2.3 Output

```
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
Item di depan 10
Item paling belakang 40
tampilan queue
 10 <--  20 <--  30 <--
10 dihapus dari queue
item di depan:20
item dibelakang: 40
tampilan queue setelah satu data dihapus
 20 <--  30 <--
```

Gambar 2.2.3 Output QueueArrayDriver_2511531018.

2.3 QueueLinkedList_2511531018

2.3.1 Kode Program

```java
package pekan4_2511531018;

import java.util.LinkedList;
import java.util.Queue;

public class QueueLinkedList_2511531018 {
   public static void main(String[] args) {
         Queue<Integer> q_1018 = new LinkedList<>();
         //tambah elemen (0, 1, 2, 3, 4, 5) ke antrian
         for (int i_1018=0; i_1018<6; i_1018++)
               q_1018.add(i_1018);
         //Menampilkan isi antrian.
         System.out.println("Elemen Antrian " + q_1018);
         // Untuk menghapus kepala antrian.
         int hapus_1018=q_1018.remove();
         System.out.println("Hapus elemen = " + hapus_1018);
         System.out.println(q_1018);
         //Untuk melihat antrian terdepan
         int depan_1018=q_1018.peek();
         System.out.println("Kepala Antrian = " +
depan_1018);

         int banyak_1018=q_1018.size();
         System.out.println("Size Antrian = " + banyak_1018);
   }

}
```

Gambar 2.3.1 Kode program QueueLinkedList_2511531018.

2.3.2 Langkah Kerja

2.3.2.1 Queue<Integer> => tipe data queue (isi angka) dan new LinkedList<>() => implementasinya pakai LinkedList.

```
public static void main(String[] args) {
        Queue<Integer> q_1018 = new LinkedList<>();
```

Gambar 2.3.2.1 Membuat Queue.

2.3.2.2 Loop dari 0 sampai dengan 5.

```
for (int i_1018=0; i_1018<6; i_1018++)
                q_1018.add(i_1018);
```

Gambar 2.3.2.2 Menambahkan elemen.

2.3.2.3 LinkedList langsung bisa ditampilkan seperti array.

```
System.out.println("Elemen Antrian " + q_1018);
```

Gambar 2.3.2.3 Menampilkan isi queue.

2.3.2.4 Remove() => hapus halaman depan.

```
int hapus_1018=q_1018.remove();
        System.out.println("Hapus elemen = " +
hapus_1018);
        System.out.println(q_1018);
```

Gambar 2.3.2.4 Menghapus elemen (dequeue).

2.3.2.5 Peek() = lihat depan tanpa hapus.

```
int depan_1018=q_1018.peek();
        System.out.println("Kepala Antrian = " +
depan_1018);
```

Gambar 2.3.2.5 Melihat elemen depan.

2.3.2.6 Size() = jumlah elemen dalam queue.

```
int banyak_1018=q_1018.size();
        System.out.println("Size Antrian = " +
banyak_1018);
  }
```

Gambar 2.3.2.6 Mengetahui jumlah elemen.

2.4.3 Output

```
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

Gambar 2.3.3 Output QueueLinkedList_2511531018.

2.4 ReverseData_2511531018

2.4.1 Kode Program

```
package pekan4_2511531018;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511531018 {
   public static void main(String[] args) {
         Queue<Integer> q_1018 = new LinkedList<Integer>();
         q_1018.add(1);
         q_1018.add(2);
         q_1018.add(3); // [1, 2, 3]
         System.out.println("sebelum reverse = " + q_1018);
         Stack<Integer> s=new Stack<Integer>();
         while (!q_1018.isEmpty()) { // Q -> S
               s.push(q_1018.remove());
         }
         while (!s.isEmpty()) { //S->Q
               q_1018.add(s.pop());
         }
         System.out.println("sesudah reverse = " + q_1018);
// [3, 2, 1]
   }
}
```

Gambar 2.4.1 Kode program ReverseData_2511531018.

2.4.2 Langkah Kerja

2.4.2.1 Isi awal queue: 1 => 2 => 3, hasilnya: “sebelum reverse = [1, 2, 3].

```
Queue<Integer> q_1018 = new LinkedList<Integer>();
        q_1018.add(1);
        q_1018.add(2);
        q_1018.add(3);
System.out.println("sebelum reverse = " + q_1018);
```

Gambar 2.4.2.1 Membuat Queue.

2.4.2.2 Stack masih kosong, remove() => ambil dari depan queue, dan push() => masuk ke stack.

```
Stack<Integer> s=new Stack<Integer>();
        while (!q_1018.isEmpty()) { // Q -> S
              s.push(q_1018.remove());
```

Gambar 2.4.2.2 Membuat stack dan memindahkan queue ke stack.

2.4.2.3 Pop() => ambil dari atas stack dan add() => masuk ke belakang queue.

```
while (!s.isEmpty()) { //S->Q
                q_1018.add(s.pop());
```

Gambar 2.4.2.3 Memindahkan stack ke queue.

2.4.2.4 Menampilkan hasil.

```
System.out.println("sesudah reverse = " + q_1018);
```

Gambar 2.4.2.4 Hasil akhir.

2.4.3 Output

```
sebelum reverse = [1, 2, 3]
sesudah reverse = [3, 2, 1]
```

Gambar 2.4.3 output ReverseData_2511531018.

2.5 IterasiQueue_2511531018

2.5.1 Kode Program

```
package pekan4_2511531018;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;

public class IterasiQueue_2511531018 {
   public static void main(String[] args)
   {
         Queue<String> q_1018=new LinkedList<>();

         q_1018.add("Praktikum");
         q_1018.add("Struktur");
         q_1018.add("Data");
         q_1018.add("Dan");
         q_1018.add("Algoritma");
         Iterator<String> iterator = q_1018.iterator();
         while (iterator.hasNext()) {
               System.out.print(iterator.next() + " ");
         }
   }

}
```

Gambar 2.5.1 Kode program IterasiQueue_251131018.

2.5.2 Langkah Kerja

2.5.2.1 Queue dengan tipe string.

```
Queue<String> q_1018=new LinkedList<>();
```

Gambar 2.5.2.1 Membuat queue.

2.5.2.2 Isi queue: Praktikum => Struktur => Data => Dan => Algoritma.

```
        q_1018.add("Praktikum");
        q_1018.add("Struktur");
        q_1018.add("Data");
        q_1018.add("Dan");
        q_1018.add("Algoritma");
```

Gambar 2.5.2.2 Menambahkan elemen.

2.5.2.3 Iterator() => ambil alat buat menelusuri queue dan mulai dari elemen paling depan.

```
Iterator<String> iterator = q_1018.iterator();
```

Gambar 2.5.2.3 Membuat iterator.

2.5.2.4 hasNext() => cek masih ada elemen atau tidak dan next() => ambil elemen berikutnya.

```
while (iterator.hasNext()) {
                System.out.print(iterator.next() + "
");
```

Gambar 2.5.2.4 Proses iterasi.

2.5.5 Output

```
Praktikum Struktur Data Dan Algoritma
```

Gambar 2.5.5 Output IterasiQueue_2511531018.

2.6 Tugas

2.6.1 Kode Program AntrianLoket_2511531018

```
package TugasPekan4;

import java.util.Scanner;

public class AntrianLoket_2511531018 {
   int front_1018;
   int rear_1018;
   int size_1018;
   int max_1018;
   String queue_1018[];

   // Constructor
   public AntrianLoket_2511531018(int max) {
       this.max_1018 = max;
       front_1018 = 0;
```

```
        rear_1018 = -1;
        size_1018 = 0;
        queue_1018 = new String[max];
    }

    // cek kosong
    boolean isEmpty_1018() {
        return size_1018 == 0;
    }

    // cek penuh
    boolean isFull_1018() {
        return size_1018 == max_1018;
    }

    // tambah antrian
    void enqueue_1018(String data) {
        if (isFull_1018()) {
            System.out.println("Antrian penuh!");
            return;
        }
        rear_1018 = (rear_1018 + 1) % max_1018;
        queue_1018[rear_1018] = data;
        size_1018++;
        System.out.println("Data berhasil ditambahkan ke
antrian");
        }

    // hapus antrian
    void dequeue_1018() {
        if (isEmpty_1018()) {
            System.out.println("Antrian kosong!");
            return;
        }
        String data = queue_1018[front_1018];
        front_1018 = (front_1018 + 1) % max_1018;
        size_1018--;
        System.out.println(data + " telah dilayani");
    }

    // tampilkan antrian
    void display_1018() {
        if (isEmpty_1018()) {
            System.out.println("Antrian kosong!");
            return;
        }
        System.out.println("Isi antrian:");
        int i_1018 = front_1018;
        int no = 1;
        for (int count = 0; count < size_1018; count++) {
            System.out.println(no + ". " + queue_1018[i_1018]);
            i_1018 = (i_1018 + 1) % max_1018;
            no++;
        }
    }

    // reverse antrian
```

```
    void reverse_1018() {
        if (isEmpty_1018()) {
            System.out.println("Antrian kosong!");
            return;
        }

        String temp_1018[] = new String[size_1018];
        int i_1018 = front_1018;

        // salin ke array sementara
        for (int j_1018 = 0; j_1018 < size_1018; j_1018++) {
            temp_1018[j_1018] = queue_1018[i_1018];
            i_1018 = (i_1018 + 1) % max_1018;
        }

        // balik isi
        front_1018 = 0;
        rear_1018 = size_1018 - 1;
        for (int j_1018 = 0; j_1018 < size_1018; j_1018++) {
            queue_1018[j_1018] = temp_1018[size_1018 - j_1018 -
1];
        }

        System.out.println("Antrian berhasil dibalik!");
    }

    // MAIN PROGRAM
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AntrianLoket_2511531018 antrian_1018 = new
AntrianLoket_2511531018(10);

        int pilih_1018;

        do {
            System.out.println("\n=== PROGRAM ANTRIAN LOKET
===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
            pilih_1018 = sc.nextInt();
            sc.nextLine();

            switch (pilih_1018) {
                case 1:
                    System.out.print("Masukkan nama pelanggan:
");
                    String nama = sc.nextLine();
                    antrian_1018.enqueue_1018(nama);
                    break;

                case 2:
                    antrian_1018.dequeue_1018();
                    break;
```

```
                case 3:
                    antrian_1018.display_1018();
                    break;

                case 4:
                    antrian_1018.reverse_1018();
                    break;

                case 5:
                    System.out.println("Program selesai");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }

        } while (pilih_1018 != 5);

        sc.close();
    }

}
```

Gambar 2.6.1 Kode Program AntrianLoket_2511531018.

2.6.2 Output

1. Menambahkan antrian.

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: syasya
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: halwa
Data berhasil ditambahkan ke antrian
```

Pada gambar diatas menunjukkan menu no.1 berguna untuk menambahkan antrian dengan memasukkan nama pelanggan ke

dalam antrian. Setelah data ditambahkan maka data berhasil ditambahkan ke antrian.

2. Menampilkan antrian.

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. syasya
2. halwa
```

Untuk menu no.3 berfungsi untuk menampilkan antrian yang telah ditambahkan.

3. Reverse.

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Antrian berhasil dibalik!
```

Menu no.4 membuat antrian menjadi terbalik. Contoh : syasya => kinaya => annisa => haliya. Ketika di reverse menjadi: haliya => annisa => kinaya => halwa.

4. Menghapus antrian.

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
halwa telah dilayani
```

Menghapus antrian berarti pelanggan telah dilayani, sehingga pelanggan tersebut keluar dari antrian.

5. Keluar

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
Program selesai
```

Tombol keluar berguna untuk user keluar dari program.

BAB III PENUTUP

Struktur data Queue (antrian) dengan prinsip FIFO (First In First Out) menggunakan berbagai pendekatan, yaitu array dan LinkedList. Melalui beberapa latihan, dipahami operasi dasar queue seperti enqueue, dequeue, pengecekan kondisi antrian, serta penampilan data. Selain itu, dikembangkan pula konsep tambahan seperti proses iterasi dan reverse antrian menggunakan bantuan stack. Pada tugas akhir, implementasi queue menggunakan array dikembangkan menjadi program antrian loket yang interaktif dengan menu, sehingga dapat mensimulasikan proses pelayanan secara lebih nyata. Dengan demikian, praktikum ini membantu dalam memahami konsep, cara kerja, serta penerapan queue dalam pemrograman secara sistematis.

DAFTAR PUSTAKA

[1] Robert Lafore, *Data Structures and Algorithms in Java*, 2nd ed. Indianapolis, IN, USA: Sams Publishing, 2003.

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.